

Unidad 13: Despliegue de Aplicaciones Python y Gestión de Usuarios

Edición de Usuario

Para editar un usuario en Django, seguiremos estos pasos: creación de la vista, la URL y el template necesario para modificar los atributos del usuario.



1. Creación de la Vista

Primero, creamos una vista que se encargará de manejar la edición del usuario. Esta vista usará `UserChangeForm`, que es un formulario de Django diseñado para permitir la edición de los atributos del usuario.

```
```python
from django.contrib.auth.forms import UserChangeForm
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
```

@login\_required # con este decorador exigimos que el usuario esté logueado para utilizar esta view

```
def editarPerfil(request):
 if request.method == 'POST':
 form = UserChangeForm(request.POST, instance=request.user)
 if form.is_valid():
 form.save()
 return redirect('perfil')
 else:
 form = UserChangeForm(instance=request.user)
 return render(request, 'editarPerfil.html', {'form': form})
```

...

---

## 2. Creación de la URL

A continuación, creamos la URL que apuntará a nuestra vista de edición de perfil. Esto se define en `urls.py`.

```
``python
from django.urls import path
from .views import editarPerfil

urlpatterns = [
 path('editarPerfil/', editarPerfil, name='EditarPerfil'),
]
```

---

## 3. Creación del Template

El template `editarPerfil.html` será el encargado de renderizar el formulario para la edición del perfil del usuario.

```
``html
{% extends "base_generic.html" %}

{% block content %}
<h2>Editar Perfil</h2>
<form method="post">
 {% csrf_token %}
 {{ form.as_p }}
 <button type="submit">Guardar Cambios</button>
</form>
{% endblock %}
```

'''

---

## 4. Uso de UserChangeForm

El formulario `UserChangeForm` se utiliza para editar los atributos del usuario. Este formulario se importa desde `django.contrib.auth.forms`.

```
'''python
from django.contrib.auth.forms import UserChangeForm

class EditProfileForm(UserChangeForm):
 class Meta:
 model = User
 fields = ('email', 'first_name', 'last_name', 'password')
 labels = {
 'email': 'Correo Electrónico',
 'first_name': 'Nombre',
 'last_name': 'Apellido',
 }
'''
```

---

## 5. Detalles del Formulario

En el formulario personalizado `EditProfileForm`, definimos los campos que queremos que el usuario pueda editar. En este caso, estamos permitiendo la edición del correo electrónico, nombre, apellido y contraseña.

---

## Conclusión

Con estos pasos, hemos creado un sistema básico para editar los atributos de un usuario en Django. Este sistema incluye la vista, la URL y el template necesarios para realizar la edición del perfil de usuario. Utilizando `UserChangeForm`, podemos modificar fácilmente los atributos del usuario y asegurarnos de que los cambios se guarden correctamente en la base de datos.

---

## Gestión de Avatares

Para gestionar avatares en una aplicación Django, seguiremos los siguientes pasos: creación del modelo Avatar, configuración de almacenamiento de imágenes, ajustes en [settings.py](#), [urls.py](#), y la implementación de vistas y templates para el manejo de avatares.

## 1. Creación del Modelo Avatar

Primero, creamos un modelo llamado `Avatar` que estará vinculado al modelo `User` de Django. Este modelo tendrá un campo para almacenar la imagen del avatar.

```
```python
from django.db import models
from django.contrib.auth.models import User

class Avatar(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    imagen = models.ImageField(upload_to='avatares', null=True, blank=True)

    def __str__(self):
        return f"{self.user.username} - {self.imagen}"
```
```

---

## 2. Configuración de Almacenamiento de Imágenes

Necesitamos configurar Django para que pueda manejar la carga y el almacenamiento de imágenes. Para ello, ajustamos la configuración en `settings.py`.

```
```python
import os

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```
```

También debemos asegurarnos de que tenemos la librería `Pillow` instalada, ya que es necesaria para manejar las imágenes en Django. Podemos instalarla usando:

```
```
pip install Pillow
```
```

---

### 3. Ajustes en urls.py

Debemos añadir las rutas necesarias en `urls.py` para servir los archivos de medios durante el desarrollo.

```
```python
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # otras rutas
]

if settings.DEBUG: # hacemos esta validación para que servir los estaticos desde django
    solo en desarrollo
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```
```

---

### 4. Creación de la Vista para Gestionar Avatares

Creamos una vista que permita a los usuarios cargar y actualizar sus avatares. Esta vista utilizará un formulario que incluye el campo de imagen.

```
```python
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .forms import AvatarForm

@login_required
def upload_avatar(request):
    if request.method == 'POST':
        form = AvatarForm(request.POST, request.FILES, instance=request.user.avatar)
        if form.is_valid():
            form.save()
            return redirect('profile')
    else:
        form = AvatarForm(instance=request.user.avatar)
    return render(request, 'upload_avatar.html', {'form': form})

```
```

---

### 5. Creación del Formulario para el Avatar

Definimos un formulario en `forms.py` que maneje la carga de la imagen del avatar.

```
```python
from django import forms
from .models import Avatar

class AvatarForm(forms.ModelForm):
    class Meta:
        model = Avatar
        fields = ['imagen']

...

```

6. Creación del Template

Creamos un template llamado `upload\_avatar.html` para permitir a los usuarios cargar su avatar.

```
```html
{% extends "base_generic.html" %}

{% block content %}
 <h2>Subir Avatar</h2>
 <form method="post" enctype="multipart/form-data">
 {% csrf_token %}
 {{ form.as_p }}
 <button type="submit">Subir</button>
 </form>
{% endblock %}

...

```

---

## Conclusión

Con estos pasos, hemos implementado la gestión de avatares en una aplicación Django. Hemos cubierto la creación del modelo `Avatar`, la configuración del almacenamiento de imágenes, los ajustes necesarios en `settings.py` y `urls.py`, y la implementación de una vista y un template para permitir a los usuarios cargar y actualizar sus avatares. Este sistema permite una gestión eficiente de avatares en una aplicación Django.

---

## Despliegue en Heroku

Heroku es una plataforma en la nube que permite a los desarrolladores desplegar, gestionar y escalar aplicaciones sin preocuparse por la infraestructura subyacente. A continuación, se describen los pasos para configurar y desplegar una aplicación Django en Heroku, así como sus ventajas y desventajas.

# 1. Introducción a Heroku

Heroku es conocido por su facilidad de uso y su capacidad para manejar aplicaciones web de manera eficiente. Ofrece una serie de servicios como bases de datos, seguridad, monitoreo y más, lo que permite a los desarrolladores centrarse en la construcción de sus aplicaciones.

## 2. Configuración Inicial

Antes de desplegar una aplicación Django en Heroku, es necesario asegurarse de tener instalada la herramienta de línea de comandos (CLI) de Heroku y tener una cuenta de Heroku.

### 1. Instalar Heroku CLI:

```
...
curl <https://cli-assets.heroku.com/install.sh> | sh
...
```

### 2. Iniciar sesión en Heroku:

```
...
heroku login
...
```

---

## 3. Preparación de la Aplicación Django

Para que una aplicación Django funcione en Heroku, se deben realizar algunos ajustes en el proyecto.

### 1. Crear un archivo `Procfile`: Este archivo le dice a Heroku cómo ejecutar la aplicación.

```
...
web: gunicorn myproject.wsgi
```

```
'''
```

**2. Instalar Gunicorn:** Gunicorn es un servidor WSGI que Heroku utiliza para ejecutar aplicaciones Django.

```
'''
```

```
pip install gunicorn
```

```
'''
```

**3. Configurar `requirements.txt`:** Asegurarse de que todos los paquetes necesarios estén listados en `requirements.txt`.

```
'''
```

```
pip freeze > requirements.txt
```

```
'''
```

**4. Configurar `runtime.txt`:** Especificar la versión de Python.

```
'''
```

```
echo "python-3.8.10" > runtime.txt
```

```
'''
```

**5. Configurar `settings.py`:**

- **Configurar la base de datos:** Utilizar `dj-database-url` para manejar la base de datos de Heroku.

```
'''python
import dj_database_url
DATABASES['default'] = dj_database_url.config(conn_max_age=600, ssl_require=True)
```

```
'''
```

- **Permitir todos los hosts:**

```
'''python
ALLOWED_HOSTS = ['*']
```

```
'''
```

- **Configurar el almacenamiento de archivos estáticos:**

```
'''python
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

```
'''
```



## 6. Crear y aplicar migraciones:

```
'''
python manage.py makemigrations
python manage.py migrate
'''
```

---

## 4. Despliegue en Heroku

### 1. Inicializar un repositorio Git (si no se ha hecho ya):

```
'''
git init
git add .
git commit -m "Initial commit"
'''
```

### 2. Crear una aplicación en Heroku:

```
'''
heroku create my-django-app
'''
```

### 3. Desplegar la aplicación:

```
'''
git push heroku master
'''
```

### 4. Configurar variables de entorno: Configurar la clave secreta de Django y otros parámetros necesarios.

```
'''
heroku config:set SECRET_KEY='your_secret_key'
heroku config:set DEBUG=False
'''
```

### 5. Escalar la aplicación para usar un web dyno:

```
'''
```

```
heroku ps:scale web=1
```

```
'''
```

---

## 5. Ventajas y Desventajas de Heroku

### Ventajas:

- **Facilidad de Uso:** Heroku simplifica el despliegue y gestión de aplicaciones.
- **Escalabilidad:** Permite escalar aplicaciones con facilidad.
- **Integraciones:** Heroku ofrece numerosos complementos y servicios que facilitan el desarrollo.
- **Ahorro de Tiempo:** Minimiza la necesidad de gestionar infraestructura, permitiendo a los desarrolladores centrarse en el código.

### Desventajas:

- **Costos:** Los servicios avanzados pueden ser costosos.
  - **Dependencia:** Alta dependencia de la plataforma para la gestión y despliegue de aplicaciones.
- 

## Conclusión

Desplegar aplicaciones Django en Heroku es un proceso relativamente sencillo que ofrece numerosas ventajas en términos de facilidad de uso y escalabilidad.

---

## Despliegue en PythonAnywhere

PythonAnywhere es una plataforma de alojamiento web que se especializa en aplicaciones basadas en Python. Ofrece un entorno amigable, fácil de usar y está diseñado para que los desarrolladores puedan desplegar aplicaciones rápidamente sin necesidad de preocuparse por la infraestructura. A continuación, se detallan los pasos para desplegar una aplicación Python en PythonAnywhere, sus características y cómo se compara con Heroku.



---

## 1. Introducción a PythonAnywhere

PythonAnywhere proporciona un entorno de desarrollo y despliegue completamente en línea. Puedes escribir tu código, ejecutarlo y desplegarlo sin necesidad de configurar servidores locales.

## 2. Características de PythonAnywhere

### Entorno Web Completo:

Acceso a un terminal Bash, un editor de código en línea y la capacidad de gestionar bases de datos.

### Soporte para Múltiples Versiones de Python:

Desde versiones antiguas hasta las más recientes.

### Fácil Integración con Git:

Permite la clonación y el despliegue de repositorios directamente desde GitHub o cualquier otro servicio Git.

### Escalabilidad:

Ofrece planes que permiten escalar tus aplicaciones según sea necesario.

---

## 3. Ventajas de PythonAnywhere

### **Fácil de Usar:**

Interfaz intuitiva y amigable para desarrolladores de todos los niveles.

### **Configuración Rápida:**

Permite un despliegue rápido sin complicaciones.

### **Plan Gratuito:**

Adecuado para proyectos pequeños y pruebas.

### **Soporte Integrado para Python:**

Muchas herramientas y bibliotecas de Python preinstaladas.

---

## **4. Comparación con Heroku**

### **Facilidad de Uso:**

Ambos son fáciles de usar, pero PythonAnywhere está más enfocado en Python, lo que puede ser una ventaja para proyectos específicos de este lenguaje.

### **Escalabilidad:**

Heroku ofrece más opciones para aplicaciones que necesitan escalar rápidamente y manejar mucho tráfico.

### **Costos:**

Ambos tienen planes gratuitos, pero los costos pueden variar según los requisitos del proyecto. Heroku puede ser más costoso para proyectos grandes.

### **Documentación y Comunidad:**

Heroku tiene una comunidad más grande y más documentación disponible.

---

## **5. Guía de Despliegue en PythonAnywhere**

Para desplegar una aplicación Django en PythonAnywhere, sigue estos pasos:

### **1. Crear una Cuenta en PythonAnywhere**

Regístrate en [PythonAnywhere](https://pythonanywhere.com) y crea una cuenta.

## 2. Configurar el Entorno

### 1. Clonar el Repositorio:

- Usa el terminal integrado de PythonAnywhere para clonar tu repositorio.

...

```
git clone <https://github.com/tu_usuario/tu_repositorio.git>
cd tu_repositorio
```

...

### 2. Crear y Activar un Entorno Virtual:

- Crea un entorno virtual para tu proyecto.

...

```
python3 -m venv myenv
source myenv/bin/activate
```

...

### 3. Instalar Dependencias:

- Instala las dependencias de tu proyecto listadas en `requirements.txt`.

...

```
pip install -r requirements.txt
```

...

---

## 3. Configurar la Base de Datos

### 1. Configurar la Base de Datos:

- Ajusta las configuraciones de la base de datos en `settings.py` si es necesario. PythonAnywhere proporciona bases de datos MySQL y PostgreSQL.

## 4. Configurar el Servidor Web

### 1. Configurar WSGI:

- PythonAnywhere utiliza WSGI para servir aplicaciones web. Configura el archivo WSGI para tu aplicación Django.

```
```python
```

```
import os
import sys

path = '/home/tu_usuario/tu_proyecto'
if path not in sys.path:
    sys.path.append(path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'tu_proyecto.settings'

from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()

'''
```

2. Configurar Archivos Estáticos:

- Asegúrate de que los archivos estáticos estén correctamente configurados en `settings.py`.

```
'''python
STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

'''
```

5. Desplegar la Aplicación

1. Aplicar Migraciones:

- Ejecuta las migraciones de la base de datos.

```
'''

python manage.py migrate

'''
```

2. Recopilar Archivos Estáticos:

- Recopila los archivos estáticos.

```
'''

python manage.py collectstatic

'''
```

3. Reiniciar el Servidor Web:

- Desde el panel de control de PythonAnywhere, reinicia el servidor web para aplicar los cambios.

Conclusión

La elección entre Heroku y PythonAnywhere depende de las necesidades específicas del proyecto.

Heroku es ideal para proyectos que requieren alta escalabilidad, múltiples servicios integrados y un rendimiento superior, aunque puede resultar más costoso. Es una excelente opción para aplicaciones complejas y con un alto volumen de tráfico.

PythonAnywhere es más adecuado para proyectos pequeños y específicos de Python, con un entorno de desarrollo amigable y planes económicos. Es una buena opción para desarrolladores que buscan un entorno sencillo y rápido para desplegar aplicaciones Python con menos tráfico.

Ambas plataformas ofrecen planes gratuitos que permiten probar sus servicios antes de comprometerse con un plan pago, lo cual es útil para tomar una decisión informada.

Pruebas Unitarias en Python

Las pruebas unitarias son una parte fundamental del desarrollo de software, ya que permiten verificar que las distintas unidades o componentes del código funcionen correctamente. En el contexto de Django, las pruebas unitarias ayudan a asegurar que las vistas, modelos y otras partes de la aplicación se comporten como se espera. A continuación, se detalla la importancia de las pruebas unitarias, cómo implementarlas en proyectos Django, ejemplos de casos de prueba y algunas mejores prácticas.

1. Introducción a las Pruebas Unitarias

Las pruebas unitarias son pequeñas pruebas automatizadas que validan el comportamiento de una unidad específica del código, como una función o un método. Estas pruebas son cruciales para:

- **Detectar Errores Tempranos:** Ayudan a encontrar errores en fases tempranas del desarrollo.
 - **Asegurar la Calidad del Código:** Mantienen la calidad del código y previenen regresiones.
 - **Facilitar Refactorizaciones:** Permiten modificar el código con confianza, sabiendo que las pruebas asegurarán que todo sigue funcionando.
-

2. Importancia de las Pruebas Unitarias

Confiabilidad: Incrementan la confiabilidad del software al asegurar que cada componente funciona correctamente.

Mantenimiento: Facilitan el mantenimiento del código a largo plazo.

Documentación: Sirven como documentación del comportamiento esperado de las funciones y métodos.

Desarrollo Ágil: Facilitan la adopción de metodologías ágiles, permitiendo pruebas continuas durante el desarrollo.

3. Implementación de Pruebas Unitarias en Django

Django proporciona un marco de pruebas integrado que facilita la creación y ejecución de pruebas unitarias. Aquí se muestra cómo implementar pruebas unitarias en un proyecto Django.

Configuración Inicial

1. Crear un archivo de pruebas:

Por convención, las pruebas se colocan en un archivo llamado `tests.py` dentro de cada aplicación Django.

2. Importar las herramientas de prueba:

```
```python
from django.test import TestCase
from .models import MiModelo

...

```

---

## Ejemplo de Caso de Prueba

A continuación se muestra un ejemplo de un caso de prueba para un modelo Django.

```
```python
from django.test import TestCase
from .models import MiModelo

class MiModeloTestCase(TestCase):
    def setUp(self):
        MiModelo.objects.create(campo1="valor1", campo2="valor2")
        MiModelo.objects.create(campo1="valor3", campo2="valor4")

    def test_mimodelo_str(self):

```



```
objeto1 = MiModelo.objects.get(campo1="valor1")
objeto2 = MiModelo.objects.get(campo1="valor3")
self.assertEqual(str(objeto1), "valor1")
self.assertEqual(str(objeto2), "valor3")
```

```
'''
```

Ejecución de las Pruebas

Para ejecutar las pruebas, se utiliza el comando:

```
'''
```

```
python manage.py test
```

```
'''
```

4. Mejores Prácticas para Pruebas Unitarias

Independencia: Cada prueba debe ser independiente de las demás.

Nombres Claros: Nombrar las pruebas de manera descriptiva para indicar claramente qué están verificando.

Cobertura de Código: Asegurarse de que las pruebas cubran la mayor cantidad de código posible, incluyendo casos extremos.

Uso de setUp y tearDown: Utilizar los métodos `setUp` y `tearDown` para preparar el entorno antes de cada prueba y limpiar después de cada prueba, respectivamente.

Pruebas Rápidas: Las pruebas deben ejecutarse rápidamente para facilitar la integración continua.

5. Ejemplos Adicionales de Casos de Prueba

Pruebas de Vistas:

```
'''python
from django.test import TestCase
from django.urls import reverse

class VistaTestCase(TestCase):
    def test_vista_responde_200(self):
        response = self.client.get(reverse('nombre_de_la_vista'))
        self.assertEqual(response.status_code, 200)

'''
```

- **Pruebas de Formularios:**

```
```python
from django.test import TestCase
from .forms import MiFormulario

class FormularioTestCase(TestCase):
 def test_formulario_valido(self):
 form_data = {'campo1': 'valor1', 'campo2': 'valor2'}
 form = MiFormulario(data=form_data)
 self.assertTrue(form.is_valid())

```
```

Conclusión

Las pruebas unitarias son esenciales para asegurar la calidad y confiabilidad del software. Implementarlas en proyectos Django es un proceso relativamente sencillo gracias al marco de pruebas integrado que ofrece Django. Siguiendo las mejores prácticas y escribiendo pruebas detalladas, los desarrolladores pueden garantizar que su código funcione correctamente y sea mantenible a largo plazo.

Material complementario:

Recursos complementarios

Archivos complementarios

- [Casos de prueba](#)
- [editarPerfil views](#)
- [editarPerfil](#)
- [-Python Tutorial](#) | W3 Schools
- [AB_NYC_2019](#)
- [Manipulación de Datos con Pandas](#)